

Mapping the morphology-conditioned PPO+AMP formulation to code in the hhi repository

Repository structure relevant to the formulation

The implementation corresponding to your CMDP + PPO + AMP (with morphology conditioning) is split across:

- **Environment / dynamics / rewards / AMP features** in `ase/env/tasks/`, primarily `base_task.py`, `humanoid.py`, and `humanoid_phc.py`. ¹
- **Policy / value / discriminator networks** in `ase/learning/`, primarily `phc_network_builder.py` (FiLM-conditioned actor, critic, and discriminator), and `phc_models.py` (routing discriminator branches in the model forward). ²
- **PPO losses and GAE** in `ase/learning/common_agent.py`, and **joint actor-critic-discriminator training** in `ase/learning/phc_agent.py`. ³
- **AMP “standard” reward shaping and reward mixing reference** exists in `ase/learning/amp_agent.py`; `phc_agent.py` follows the same AMP structure but adds *shape conditioning throughout the discriminator* and carries `amp_shape` tensors end-to-end. ⁴

(Throughout this report, the code is hosted on GitHub ⁵ . ⁶)

Formula-to-code correspondences

Morphology-Conditioned MDP

Math component.

You define a conditional MDP with morphology code $m = [\text{gender}, \beta_1, \dots, \beta_{10}]$, dynamics $P(s_{t+1} \mid s_t, a_t, m)$, and a morphology-conditioned policy $\pi_\theta(a_t \mid s_t, m)$.

Where morphology m is constructed and attached to each environment instance.

`ase/env/tasks/humanoid.py` loads gender-beta keys from motion filenames, constructs a per-template vector that concatenates a gender encoding with SMPL betas, and then assigns a template's m to each simulated environment (`self._betas_env[i] = ...`) while recording a per-env key map (`env_id_beta_keys_map`). ⁷

Code (abridged, key operations).

```
# humanoid.py (asset/template setup)
gender_tensor = encode_gender(gender)
curr_gender_beta = cat([gender_tensor, betas]) # m ∈ R^{11}
template_betas.append(curr_gender_beta)

# per-env assignment
asset_idx = i % num_templates
```

```
env_id_beta_keys_map[i] = (gender, beta_key)
_betas_env[i] = _template_betas[asset_idx]
```

7

Where morphology is injected into the state given to the policy.

`ase/env/tasks/humanoid_phc.py` appends `self._betas_env` (explicitly commented as `[num_envs, 11]`) to the observation buffer, making the policy input effectively (s_t, m) even though the code passes a single concatenated tensor. ⁸

Code (abridged).

```
# humanoid_phc.py
obs = compute_state_task_obs(...)
betas = _betas_env[env_ids] if env_ids else _betas_env
obs = cat([obs, betas], dim=-1) # obs := [state/task, m]
```

8

Where transition dynamics $s_{t+1} \sim P(\cdot | s_t, a_t, m)$ are realized.

The *physics transition* is implemented in `BaseTask.step`: it applies actions (`pre_physics_step(actions)`), advances the simulator (`gym.simulate`), then computes observations/rewards/dones in `post_physics_step`. ⁹
Actions are converted to PD targets or torques/forces in `Humanoid.pre_physics_step`. ¹⁰

Code (abridged).

```
# base_task.py
def step(actions):
    pre_physics_step(actions) # apply a_t
    _physics_step()          # simulate → s_{t+1}
    post_physics_step()      # compute obs, reward, done

# humanoid.py
def pre_physics_step(actions):
    if pd_control:
        set_dof_position_target_tensor(action_to_pd_targets(actions))
    else:
        set_dof_actuation_force_tensor(actions * motor_efforts * power_scale)
```

11

Where $\pi_\theta(a_t | s_t, m)$ is explicitly implemented (conditioning mechanism).

`ase/learning/phc_network_builder.py` splits the observation into a “state/task” part and an 11D “gender_betas” part, and uses the latter to generate FiLM parameters that modulate the actor MLP. This is the most direct code-realization of $\pi_\theta(\cdot | s_t, m)$. ¹²

Code (abridged).

```
# phc_network_builder.py
state_obs = obs[:, :574]
gender_beta = obs[:, 574:]          #  $m \in \mathbb{R}^{11}$ 

film = cond_linear(cond_mlp(gender_beta))
a_h = forward_with_film(actor_mlp, actor_cnn(state_obs), film)

(mu, sigma) = action_head(a_h)      #  $\pi_\theta$  for continuous control
```

12

Learning objective and reward decomposition

Math component.

Maximize expected discounted return, with per-step reward

$$r_t = r_{\text{task}}(s_t, a_t, m) + r_{\text{AMP}}(x_t, m).$$

Where r_{task} is computed in the environment.

In `HumanoidPHC.post_physics_step`, the task reward is computed by `_compute_reward(...)`, which fills `self.rew_buf`. 13

Code (abridged).

```
# humanoid_phc.py
_compute_observations(task_obs=...)
_compute_reward(ref_body_pos, ref_body_rot, ref_body_vel, ref_body_ang_vel)
rew_buf[:] = imitation_reward(...) + optional_power_penalty
```

13

Where AMP features x_t are produced.

Also in `post_physics_step`, the environment builds AMP observation windows and exports them through `extras["amp_obs"]`. It exports `extras["amp_shape"] = self._betas_env` so the discriminator can condition on the same m . 14

Code (abridged).

```
# humanoid_phc.py
_update_hist_amp_obs()
_compute_amp_observations()
extras["amp_obs"] = amp_obs_flat
extras["amp_shape"] = _betas_env      #  $m \in \mathbb{R}^{11}$ 
```

14

Where r_{AMP} is added to the RL reward.

The training code computes AMP rewards after rollout and combines them with task rewards. In `phc_agent.py`, the rollout tensors include `mb_rewards` (task reward) and AMP tensors, then AMP rewards are computed and merged before GAE/returns are computed. ¹⁵

The standard *explicit* reward mixing rule is shown in `amp_agent.py`:

$$r = w_{\text{task}} r_{\text{task}} + w_{\text{disc}} r_{\text{disc}}.$$

¹⁶

Code (abridged).

```
# amp_agent.py (reference implementation)
disc_r = amp_rewards["disc_rewards"]
combined = task_reward_w * task_rewards + disc_reward_w * disc_r
```

¹⁶

Value function learning and Bellman-style bootstrapping

Math component.

A morphology-conditioned value function $V_{\phi}(s_t, m)$ with bootstrap target $y_t = r_t + \gamma V_{\phi}(s_{t+1}, m)$ and typically squared loss.

Where the critic computes $V(s, m)$.

`phc_network_builder.py` conditions the critic on the same `gender_betas` (11D morphology code) via FiLM, then outputs a scalar value head. ¹⁷

Code (abridged).

```
# phc_network_builder.py
state_obs = obs[:, :574]
gender_beta = obs[:, 574:]

film = critic_cond_linear(critic_cond_mlp(gender_beta))
c_h = forward_with_film(critic_mlp, critic_cnn(state_obs), film)

V = value_head(c_h)
```

¹⁷

Where the one-step temporal-difference form $\delta_t = r_t + \gamma V_{t+1} - V_t$ appears.

`common_agent.py::discount_values` computes `delta = reward + gamma * next_value - value` inside the reversed-time GAE loop, which is exactly the TD residual used in GAE (your δ_t). ¹⁸

Code (abridged).

```
# common_agent.py
delta = rewards[t] + gamma * next_values[t] - values[t] # TD residual
last = delta + gamma * tau * not_done * last # GAE recursion
advs[t] = last
```

18

Where the critic regression loss corresponds to $L_V(\phi) = \mathbb{E}[(V - y)^2]$.

`common_agent.py::_critic_loss` computes squared error against `return_batch` and optionally applies PPO-style value clipping. 18

Code (abridged).

```
# common_agent.py
if clip_value:
    v_clip = old_v + clamp(v - old_v, -eps, +eps)
    c_loss = max((v - ret)^2, (v_clip - ret)^2)
else:
    c_loss = (ret - v)^2
```

18

Advantage estimation and returns

Math component.

GAE advantage

$$A_t = \sum_{l \geq 0} (\gamma \lambda)^l \delta_{t+l}, \quad R_t = A_t + V(s_t, m).$$

Where advantages and returns are constructed.

`phc_agent.py` uses `discount_values(...)` to compute `mb_advs`, then sets `mb_returns = mb_advs + mb_values`. This is the same relationship $R_t = A_t + V_t$. 15

Code (abridged).

```
# phc_agent.py
mb_advs = discount_values(dones, values, rewards, next_values) # GAE(γ, τ)
mb_returns = mb_advs + mb_values
# R = A + V
```

19

PPO actor objective

Math component.

Clipped PPO surrogate:

$$L_{\text{PPO}}(\theta) = \mathbb{E}[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t)].$$

Where the ratio-and-clip surrogate is computed.

`common_agent.py::_actor_loss` computes the likelihood ratio (via exponentiated log-prob differences), constructs two surrogates, and takes the max of negative surrogates (equivalently minimizing $-L_{\text{PPO}}$). ¹⁸

Code (abridged).

```
# common_agent.py
ratio = exp(old_logp - logp)
s1 = adv * ratio
s2 = adv * clamp(ratio, 1 - eps, 1 + eps)
actor_loss = max(-s1, -s2)
```

¹⁸

Where it is used in joint optimization.

`phc_agent.py::calc_gradients` calls `_actor_loss(...)` and then aggregates (masked) actor loss into the final training loss. ²⁰

Adversarial motion learning: discriminator $D_\psi(x, m)$

Math component.

A discriminator $D_\psi(x, m)$ trained to classify real vs generated motion windows, where conditioning on m is explicit in your formulation.

Where the discriminator network is defined and conditioned on morphology.

`phc_network_builder.py` builds a discriminator MLP (`_disc_mlp` + `_disc_logits`) and a morphology conditioner network (`disc_cond_mlp` + `disc_cond_linear`) that produces FiLM parameters from `amp_shape` (11D). The discriminator forward `eval_disc(amp_obs, amp_shape)` applies FiLM to the discriminator trunk and outputs a discriminator **logit**. ²¹

Code (abridged).

```
# phc_network_builder.py
cond = disc_cond_linear(disc_cond_mlp(amp_shape)) # from m ∈ R^{11}
film = split_to_gamma_beta(cond, disc_hidden_units)

h = forward_with_film(disc_mlp, amp_obs, film)
logit = disc_logits(h) # Dψ returns logits
```

²¹

Where the three discriminator “branches” (rollout, replay, demo) are evaluated.

`phc_models.py` extends the standard rl-games model forward to additionally compute `disc_agent_logit`, `disc_agent_replay_logit`, and `disc_demo_logit` by calling `a2c_network.eval_disc(...)` with the corresponding `(amp_obs, amp_shape)` pairs. ²²

Discriminator objective $L_D(\psi)$

Math component.

Binary classification loss:

$$L_D(\psi) = -\mathbb{E}_{x \sim p_{\text{data}}}[\log D(x, m)] - \mathbb{E}_{x \sim p_{\theta}}[\log(1 - D(x, m))].$$

Where the BCE discriminator loss is implemented.

`phc_agent.py::_disc_loss` computes:

- BCE-with-logits loss that pushes *fake logits* $\rightarrow 0$ and *real logits* $\rightarrow 1$ (`_disc_loss_neg`, `_disc_loss_pos`);
- plus regularizers (logit L2, gradient penalty on demo samples, optional weight decay). ²³

Code (abridged).

```
# phc_agent.py
loss_fake = BCEWithLogits(fake_logits, zeros)
loss_real = BCEWithLogits(real_logits, ones)
disc_loss = 0.5 * (loss_fake + loss_real) + regularizers
```

²³

AMP reward $r_{\text{AMP}}(x, m)$

Math component.

Typical AMP shaping uses discriminator output to define a style reward, e.g.

$$r_{\text{AMP}}(x, m) = -\log(1 - D(x, m)),$$

up to conventions.

Where the canonical AMP shaping is implemented.

In `amp_agent.py::_calc_disc_rewards`, the discriminator returns logits; the code converts logits to a probability via a sigmoid and then uses the $-\log(1 - D)$ form with a small epsilon floor, followed by scaling by `disc_reward_scale`. ²⁴

Code (abridged).

```
# amp_agent.py
prob = sigmoid(disc_logits)
```

```
r_amp = -log(max(1 - prob, 1e-4))
r_amp *= disc_reward_scale
```

24

Where morphology enters $r_{\text{AMP}}(x, m)$ in PHC.

In the PHC path, the discriminator logits depend on both `(amp_obs, amp_shape)` through `eval_disc(amp_obs, amp_shape)`, so the same reward shaping becomes $r_{\text{AMP}}(x, m)$ by composition. ²⁵

Replay buffer mixture p_{fake}

Math component.

You describe a “fake” distribution as a mixture of current rollouts and replay buffer samples:

$$p_{\text{fake}}(x \mid m) = \alpha p_{\pi}(x \mid m) + (1 - \alpha) p_{\text{replay}}(x \mid m).$$

Where the replay-vs-rollout mixing occurs in practice.

In `phc_agent.py::train_epoch`, the training batch is augmented with:

- `amp_obs_demo`, `amp_shape_demo` sampled from the demo buffer;
- `amp_obs_replay`, `amp_shape_replay` sampled from the replay buffer (or copied from current rollout if replay is empty). ²⁶

This corresponds to training the discriminator on a mixture of (current, replay) negatives, albeit expressed procedurally rather than via an explicit α . ²⁷

Code (abridged).

```
# phc_agent.py (epoch prep)
demo = amp_obs_demo_buffer.sample(num_obs_samples)
repl = amp_replay_buffer.sample(num_obs_samples) # else fallback to rollout
batch["amp_obs_demo"], batch["amp_shape_demo"] = demo
batch["amp_obs_replay"], batch["amp_shape_replay"] = repl
```

26

Where the discriminator actually consumes the mixture.

In `phc_agent.py::calc_gradients`, fake logits are formed by concatenating the discriminator logits from the current rollout and replay batches before computing `_disc_loss`. This is an implicit mixture where the effective α is governed by relative batch sizes (here: concatenation of equal-sized minibatches implies an approximately 50/50 negative mix). ²⁰

Full training objective as a three-player optimization

Math component.

Your final objective is:

- actor: maximize task + AMP reward,

- critic: minimize value regression,
- discriminator: minimize classification loss.

Where the “joint” loss is formed and optimized.

`phc_agent.py::calc_gradients` computes:

- PPO actor loss (`a_loss`) using the clipped surrogate,
- PPO critic loss (`c_loss`) via value regression,
- discriminator loss (`disc_loss`) via `_disc_loss(...)`, then combines them into a single scalar `loss` with coefficients, and backpropagates through the full model. ²⁸

Code (abridged).

```
# phc_agent.py
loss = actor_loss + critic_coef * critic_loss - entropy_coef * entropy +
bounds_coef * bound_loss
loss += disc_coef * disc_loss

optimizer.step( grad(loss) )
```

²⁰

Notes on morphology conditioning in this codebase

Morphology conditioning is implemented **twice** and in two distinct ways that correspond closely to your formulation.

First, morphology m is part of the *MDP context* by construction: different humanoid assets (SMPL templates keyed by gender + betas) are instantiated across environments, and the per-env

`self._betas_env` is propagated through observations and AMP metadata. ²⁹

Second, morphology m is part of the *policy and discriminator parameterization* via FiLM. The actor explicitly splits the observation into state/task features and an 11D morphology vector that generates FiLM parameters for the actor MLP. ¹²

The discriminator similarly maps `amp_shape` (11D) to FiLM parameters used to modulate discriminator hidden activations in `eval_disc(amp_obs, amp_shape)`. ²¹

This corresponds to a code-level interpretation of $\pi_\theta(a \mid s, m)$ and $D_\psi(x, m)$ where m is not merely concatenated to features, but used as a *conditioning signal that modulates intermediate representations*.

³⁰

Observed divergences and implementation details relative to the math write-up

The discriminator is trained in **logit space** using `BCEWithLogitsLoss`, which is equivalent to a Bernoulli cross-entropy on $\sigma(\text{logit})$, i.e., on $D(x, m)$ as a probability. This matches your binary classification objective but is numerically more stable. ³¹

The AMP reward uses the $-\log(1 - D)$ shaping in the repository's canonical AMP implementation (`amp_agent.py`), with an epsilon floor and an overall scale `disc_reward_scale`. This corresponds to your “typical AMP reward” form rather than the $\log D$ variant. ³²

The fake distribution mixture p_{fake} is not parameterized by an explicit scalar α . Instead, fake negatives are built as a concatenation of rollout and replay discriminator minibatches (`disc_agent_cat_logits = cat(...)`), which implicitly sets mixture weights by sample counts. ³³

End-to-end interpretation in the repository's terms

Putting the pieces together, the repository learns a **morphology-conditioned physical control policy** by:

- instantiating multiple SMPL-based humanoid assets across environments and attaching their morphology code to both observations and AMP metadata, ²⁹
- training an actor and critic that condition on the same 11D morphology vector (FiLM-conditioned networks), ¹²
- training a morphology-conditioned discriminator $D_\psi(x, m)$ over motion windows, with negatives drawn from both current rollouts and a replay buffer and positives drawn from the motion dataset, ³⁴
- optimizing the combined PPO + discriminator objective in a single differentiable training step (actor-critic losses plus discriminator loss). ³⁵

¹ ⁹ ¹¹ https://github.com/hansen1416/hhi/blob/main/ase/env/tasks/base_task.py
https://github.com/hansen1416/hhi/blob/main/ase/env/tasks/base_task.py

² ¹² ¹⁷ ²¹ ²⁵ ³⁰ ³⁴ https://github.com/hansen1416/hhi/blob/main/ase/learning/phc_network_builder.py
https://github.com/hansen1416/hhi/blob/main/ase/learning/phc_network_builder.py

³ ¹⁸ https://github.com/hansen1416/hhi/blob/main/ase/learning/common_agent.py
https://github.com/hansen1416/hhi/blob/main/ase/learning/common_agent.py

⁴ ⁵ ¹⁶ ²⁴ ³² https://github.com/hansen1416/hhi/blob/main/ase/learning/amp_agent.py
https://github.com/hansen1416/hhi/blob/main/ase/learning/amp_agent.py

⁶ <https://github.com/hansen1416/hhi>
<https://github.com/hansen1416/hhi>

⁷ ¹⁰ ²⁹ <https://github.com/hansen1416/hhi/blob/main/ase/env/tasks/humanoid.py>
<https://github.com/hansen1416/hhi/blob/main/ase/env/tasks/humanoid.py>

⁸ ¹³ ¹⁴ https://github.com/hansen1416/hhi/blob/main/ase/env/tasks/humanoid_phc.py
https://github.com/hansen1416/hhi/blob/main/ase/env/tasks/humanoid_phc.py

¹⁵ ¹⁹ ²⁰ ²³ ²⁶ ²⁷ ²⁸ ³¹ ³³ ³⁵ https://github.com/hansen1416/hhi/blob/main/ase/learning/phc_agent.py
https://github.com/hansen1416/hhi/blob/main/ase/learning/phc_agent.py

²² https://github.com/hansen1416/hhi/blob/main/ase/learning/phc_models.py
https://github.com/hansen1416/hhi/blob/main/ase/learning/phc_models.py